

Advanced Encryption Standard (AES)

Aspekt	Beschreibung
Definition von AES	AES ist ein symmetrisches Verschlüsselungsverfahren, das zur Sicherung von Daten verwendet wird. AES ist in mehreren Varianten verfügbar: AES-128, AES-192 und AES-256, basierend auf der Schlüssellänge (128, 192 bzw. 256 Bit).
Schlüssellängen	AES-128: Verwendet einen 128-Bit-Schlüssel (16 Bytes). AES-192: Verwendet einen 192-Bit-Schlüssel (24 Bytes). AES-256: Verwendet einen 256-Bit-Schlüssel (32 Bytes).
Blockgröße	AES arbeitet mit einer Blockgröße von 128 Bit (16 Byte) für alle Schlüssellängen.
Modi der AES-Verschlüsselung	AES kann mit verschiedenen Betriebsmodi verwendet werden. Jeder Modus hat unterschiedliche Eigenschaften und eignet sich für unterschiedliche Einsatzszenarien. Dazu gehören: ECB (Electronic Codebook), CBC (Cipher Block Chaining), CTR (Counter), GCM (Galois/Counter Mode)
Wichtigkeit von Padding	Da AES mit festen Blockgrößen (128 Bit) arbeitet, ist Padding erforderlich, wenn die zu verschlüsselnden Daten nicht exakt 128 Bit lang sind. Es gibt verschiedene Padding-Methoden wie: PKCS7, NoPadding (keine Padding-Methode)
Java Implementierung	In Java kann AES über die <code>javax.crypto</code> -Bibliothek und die Klasse <code>Cipher</code> implementiert werden. AES wird als Teil der Java Cryptography Extension (JCE) angeboten.
Verschlüsselungsprozess (Allgemein)	1. Schlüsselgenerierung: Der geheime Schlüssel wird aus einer Passwortquelle oder einer Zufallsquelle generiert. 2. Datenaufteilung: Die zu verschlüsselnden Daten werden in Blöcke unterteilt (Padding, falls nötig). 3. Verschlüsselung: Der Block wird unter Verwendung des AES-Algorithmus verschlüsselt. 4. Speichern/Übertragen: Der verschlüsselte Text wird übertragen oder gespeichert.
Entschlüsselungsprozess (Allgemein)	1. Schlüsselbereitstellung: Der geheime Schlüssel, der bei der Verschlüsselung verwendet wurde, muss verfügbar sein. 2. Datenaufteilung: Die verschlüsselten Daten werden ebenfalls in Blöcke unterteilt. 3. Entschlüsselung: Der Block wird mit dem AES-Algorithmus entschlüsselt. 4. Padding entfernen: Bei Bedarf wird das Padding entfernt, um den Originaltext wiederherzustellen.

Aspekt	Beschreibung
Java-Klassen	<code>Cipher</code>: Hauptklasse, die die AES-Verschlüsselung und -Entschlüsselung durchführt. <code>KeyGenerator</code>: Wird verwendet, um einen geheimen Schlüssel zu erzeugen. <code>SecretKeySpec</code>: Wird verwendet, um einen Schlüssel aus einem Byte-Array zu erstellen. <code>IvParameterSpec</code>: Wird verwendet, um den Initialisierungsvektor (IV) für Modi wie CBC und CTR zu spezifizieren.
Wichtige Methoden	<code>Cipher.getInstance(String transformation)</code>: Initialisiert die Cipher-Klasse für AES. <code>Cipher.init(int mode, Key key)</code>: Setzt den Modus (Verschlüsselung oder Entschlüsselung) und den Schlüssel. <code>Cipher.doFinal(byte[] input)</code>: Führt die tatsächliche Verschlüsselung oder Entschlüsselung durch. <code>KeyGenerator.getInstance(String algorithm)</code>: Initialisiert einen Schlüsselgenerator. <code>IvParameterSpec</code>: Ermöglicht die Verwendung eines Initialisierungsvektors (für Modis wie CBC oder CTR).
Initialisierungsvektor (IV)	In einigen Modi, wie CBC, ist ein Initialisierungsvektor (IV) erforderlich. Der IV muss entweder zufällig generiert oder vom Sender und Empfänger gemeinsam verwendet werden. Er stellt sicher, dass gleiche Eingabedaten zu unterschiedlichen Ausgabedaten führen.
Schlüssellängen und Sicherheit	AES-128 bietet eine gute Sicherheitsstufe, eignet sich jedoch weniger für hochsensible Daten im Vergleich zu AES-256. AES-256 ist der sicherste Modus und wird in der Regel für die höchste Sicherheitsanforderung verwendet.
Sicherheitsaspekte	Verwende starke Schlüssel (z.B. AES-256) für höchste Sicherheit. Vermeide den ECB-Modus, da er wiederholte Muster im Text nicht verschlüsselt. Verwende sichere Schlüssellager zur Speicherung von Schlüsseln. Implementiere Key-Management-Strategien, um die Sicherheit zu gewährleisten.
Kritische Überlegungen zur Implementierung	Padding: Bei Verwendung von Modi wie CBC muss Padding angewendet werden, um sicherzustellen, dass die Datenlänge ein Vielfaches von 128 Bit beträgt. Zufälligkeit des Schlüssels: Achte darauf, dass der Schlüssel sicher und zufällig generiert wird, um Kollisionen und Angriffe zu vermeiden. IV-Verwaltung: Ein IV sollte für jeden Verschlüsselungsvorgang neu generiert und mit den verschlüsselten Daten gespeichert oder übertragen werden.
Betriebsmodi von AES (Moduswahl)	ECB (Electronic Codebook): Einfachste Form, aber unsicher für große Datenmengen (alle Blöcke werden gleich verschlüsselt). CBC (Cipher Block Chaining): Sicherer als ECB, da jeder Block mit dem vorherigen Block verknüpft wird. CTR (Counter): Arbeitet mit einem Zähler, ermöglicht parallele Verarbeitung. GCM (Galois/Counter Mode): Kombiniert Verschlüsselung und Authentifizierung, wird oft in modernen Anwendungen verwendet.

Aspekt	Beschreibung
Schlüsselmanagement	KeyStore: In Java kann ein <code>KeyStore</code> verwendet werden, um AES-Schlüssel sicher zu speichern. Key Derivation Functions (KDFs): Verwende KDFs wie PBKDF2, um aus einem Passwort einen AES-Schlüssel zu erzeugen. Rotierende Schlüssel: Um Schlüsselkompromittierungen zu vermeiden, sollten Schlüssel regelmäßig rotiert und ersetzt werden.
Leistung und Parallelität	Parallele Streams: AES kann parallelisiert werden, insbesondere bei der Verarbeitung großer Datenmengen. Hardware-Beschleunigung: Moderne Prozessoren unterstützen AES durch Hardware-Beschleunigung, was die Leistung verbessert.
Fehlerbehandlung	<code>InvalidKeyException</code> : Wird geworfen, wenn ein ungültiger Schlüssel verwendet wird. <code>BadPaddingException</code> : Wird geworfen, wenn das Padding beim Entschlüsseln nicht korrekt ist. <code>IllegalBlockSizeException</code> : Wird geworfen, wenn die Blockgröße nicht der Spezifikation entspricht.

Methoden

Methode/Klasse	Beschreibung
<code>Cipher.getInstance(String transformation)</code>	Wird verwendet, um eine Instanz der <code>Cipher</code> -Klasse zu erstellen. Die <code>transformation</code> -Angabe beschreibt den Algorithmus sowie den Betriebsmodus und das Padding. Beispiel: <code>"AES/CBC/PKCS5Padding"</code> . Diese Methode gibt eine <code>Cipher</code> -Instanz zurück, die für die Verschlüsselung oder Entschlüsselung genutzt werden kann.
<code>Cipher.init(int mode, Key key)</code>	Diese Methode wird verwendet, um die <code>Cipher</code> -Instanz zu initialisieren. Der Modus (<code>ENCRYPT_MODE</code> oder <code>DECRYPT_MODE</code>) gibt an, ob Verschlüsselung oder Entschlüsselung durchgeführt wird. Der <code>key</code> ist der geheime Schlüssel, der zur Verschlüsselung oder Entschlüsselung verwendet wird.
<code>Cipher.doFinal(byte[] input)</code>	Diese Methode führt die Verschlüsselung oder Entschlüsselung eines gegebenen Arrays von Bytes aus. Sie gibt das verschlüsselte (oder entschlüsselte) Ergebnis zurück. Wichtig: Bei der Verschlüsselung wird das Padding für unvollständige Blockgrößen angewendet, und bei der Entschlüsselung wird das Padding entfernt.
<code>KeyGenerator.getInstance(String algorithm)</code>	Erstellt einen <code>KeyGenerator</code> für einen bestimmten Algorithmus, z.B. <code>"AES"</code> . Dies ist nützlich, um einen sicheren Schlüssel zu generieren, ohne dass der Benutzer manuell einen Schlüssel angeben muss.

Methode/Klasse	Beschreibung
<code>SecretKeySpec(byte[] key, String algorithm)</code>	Wird verwendet, um einen geheimen Schlüssel aus einem gegebenen Byte-Array zu erstellen. Dies ist besonders hilfreich, wenn der Schlüssel als Byte-Array vorliegt (z.B. nach der Entschlüsselung oder aus einer Datei) und als <code>SecretKey</code> für die Verschlüsselung genutzt werden muss.
<code>IvParameterSpec(byte[] iv)</code>	Wird verwendet, um einen Initialisierungsvektor (<code>IV</code>) zu erstellen, der in den AES-Modi wie CBC oder CTR erforderlich ist. Der <code>IV</code> wird mit den verschlüsselten Daten gespeichert oder übertragen, um ihn beim Entschlüsseln korrekt zu verwenden.
<code>KeyStore</code>	Ein Java-Container zum sicheren Speichern von Schlüsseln und Zertifikaten. Der <code>KeyStore</code> schützt Schlüssel durch Passwortsicherheit und erlaubt es, verschiedene Schlüsselarten zu verwalten.